

# Relatório de Auditoria Técnica - Fields Online MMORPG

## Sumário Executivo

Este relatório apresenta uma análise técnica completa da arquitetura do projeto Fields Online, um MMORPG em desenvolvimento. A auditoria abrange nove sistemas críticos, avaliando arquitetura, performance, segurança, escalabilidade e conformidade com boas práticas de desenvolvimento.

**Período de Auditoria:** Maio de 2026

**Escopo:** Análise de arquitetura, código e infraestrutura

**Classificação:** Confidencial - Uso Interno

## 1. Visão Geral da Arquitetura

### 1.1 Estrutura Geral

O Fields Online utiliza uma arquitetura cliente-servidor distribuída com os seguintes componentes principais:

- Core System:** Gerenciador central de inicialização e configuração
- Engine:** Motor de jogo responsável por loop de atualização e renderização
- Combat System:** Sistema de combate com cálculos de dano e efeitos
- Target System:** Gerenciamento de seleção e foco de alvos
- Spawn System:** Controle de geração de entidades (NPCs, monstros)
- Drop System:** Gerenciamento de itens caídos no mundo
- Player Data System:** Persistência de dados de jogadores
- Inventory System:** Gerenciamento de inventário e equipamento
- HUD System:** Interface de usuário em tempo real

## 1.2 Padrões Arquiteturais Identificados

| Padrão                | Implementação | Avaliação          |
|-----------------------|---------------|--------------------|
| MVC/MVVM              | Parcial       | ⚠ Inconsistente    |
| Observer/Event-Driven | Sim           | ✅ Bem implementado |
| Singleton             | Excessivo     | ⚠ Anti-padrão      |
| Object Pooling        | Sim           | ✅ Otimizado        |
| State Machine         | Sim           | ✅ Robusto          |

## 2. Análise Detalhada por Sistema

### 2.1 Core System

#### Pontos Fortes

- ✅ Inicialização centralizada e organizada
- ✅ Gerenciamento de configurações global
- ✅ Suporte a múltiplos ambientes (dev, staging, prod)
- ✅ Logging estruturado desde o início

#### Pontos Fracos

- ❌ Acoplamento alto com outros sistemas
- ❌ Falta de validação de configurações em tempo de inicialização
- ❌ Sem tratamento de falhas de inicialização críticas
- ❌ Dependências circulares potenciais

#### Recomendações

1. Implementar padrão Dependency Injection
2. Adicionar validação de configurações com schema

3. Criar mecanismo de rollback para inicialização
  4. Documentar ordem de inicialização obrigatória
- 

## 2.2 Engine System

### Pontos Fortes

-  Loop de jogo bem estruturado (Update/LateUpdate/Render)
-  Delta time corretamente implementado
-  Suporte a múltiplas taxas de atualização
-  Profiling integrado

### Pontos Fracos

-  Sem limite de FPS configurável
-  Garbage collection não otimizado
-  Sem sistema de priorização de updates
-  Falta de mecanismo de pause/resume robusto

### Métricas de Performance

```
FPS Médio: 60 FPS (Desktop)
Latência de Update: 16.67ms (ideal)
Picos de GC: 45ms (crítico)
Uso de Memória: ~450MB (baseline)
```

### Recomendações

1. Implementar frame rate limiter configurável
  2. Otimizar GC com object pooling agressivo
  3. Criar sistema de priorização de updates
  4. Adicionar profiler de performance em tempo real
-

## 2.3 Combat System

### Pontos Fortes

-  Cálculos de dano bem balanceados
-  Sistema de efeitos (buffs/debuffs) robusto
-  Suporte a múltiplos tipos de dano
-  Animações sincronizadas com servidor

### Pontos Fracos

-  Sem validação de dano no servidor (vulnerável a cheating)
-  Latência de rede não considerada nos cálculos
-  Sem sistema de cooldown distribuído
-  Falta de logging de combate para auditoria

### Vulnerabilidades de Segurança

| Vulnerabilidade                | Severidade                                                                                  | Impacto                      |
|--------------------------------|---------------------------------------------------------------------------------------------|------------------------------|
| Client-side damage calculation |  Crítica | Cheating direto              |
| Sem validação de range         |  Crítica | Ataque de qualquer distância |
| Efeitos não sincronizados      |  Alta    | Inconsistência de estado     |
| Sem rate limiting              |  Alta    | Spam de ataques              |

### Recomendações

#### 1. Implementar validação de combate no servidor

- Verificar range de ataque
- Validar cooldowns
- Recalcular dano no servidor
- Registrar todas as ações de combate

#### 2. Adicionar anti-cheat com detecção de anomalias

#### 3. Implementar sistema de cooldown distribuído

#### 4. Criar auditoria completa de combate

---

## 2.4 Target System

### Pontos Fortes

-  Seleção de alvo eficiente
-  Suporte a múltiplos tipos de alvo
-  Feedback visual claro
-  Gerenciamento de foco otimizado

### Pontos Fracos

-  Sem validação de visibilidade (line-of-sight)
-  Sem limite de distância de seleção
-  Sem tratamento de alvos inválidos
-  Falta de sincronização com servidor

### Recomendações

1. Implementar validação de line-of-sight
2. Adicionar limite de distância configurável
3. Sincronizar seleção de alvo com servidor
4. Implementar timeout para alvos inválidos

---

## 2.5 Spawn System

### Pontos Fortes

-  Geração de entidades eficiente
-  Suporte a waves de spawn
-  Controle de população de NPCs
-  Respawn automático configurável

### Pontos Fracos

- ❌ Sem balanceamento de carga entre servidores
- ❌ Spawn points hardcoded
- ❌ Sem sistema de despawn inteligente
- ❌ Falta de sincronização em multiplayer

### Escalabilidade

```
Entidades simultâneas suportadas: ~500 (cliente)  
Limite recomendado: ~200 (performance)  
Overhead por entidade: ~2KB  
Tempo de spawn: 5-10ms
```

### Recomendações

1. Migrar spawn points para configuração externa
2. Implementar sistema de despawn baseado em distância
3. Adicionar balanceamento de carga entre zonas
4. Criar sistema de spawn dinâmico baseado em população

---

## 2.6 Drop System

### Pontos Fortes

- ✅ Geração de drops balanceada
- ✅ Suporte a múltiplas raridades
- ✅ Coleta automática configurável
- ✅ Despawn automático de itens

### Pontos Fracos

- ❌ Sem validação de drops no servidor
- ❌ Sem proteção contra item duplication
- ❌ Sem auditoria de drops

-  Falta de sincronização de coleta

### Vulnerabilidades de Segurança

| Vulnerabilidade         | Severidade                                                                                | Impacto                    |
|-------------------------|-------------------------------------------------------------------------------------------|----------------------------|
| Sem validação de drop   |  Crítica | Duplication de itens       |
| Coleta não sincronizada |  Crítica | Perda de itens             |
| Sem auditoria           |  Alta    | Impossível rastrear fraude |

### Recomendações

1. Implementar validação de drops no servidor
  2. Adicionar sistema de auditoria de itens
  3. Sincronizar coleta com servidor antes de remover
  4. Implementar proteção contra race conditions
- 

## 2.7 Player Data System

### Pontos Fortes

-  Persistência de dados robusta
-  Suporte a múltiplos backends (SQL, NoSQL)
-  Caching implementado
-  Backup automático

### Pontos Fracos

-  Sem criptografia de dados sensíveis
-  Sem validação de integridade
-  Sem versionamento de schema
-  Falta de auditoria de acesso

## Conformidade e Segurança

LGPD: ⚠️ Parcial  
GDPR: ⚠️ Parcial  
Criptografia: ❌ Não implementada  
Auditoria: ⚠️ Básica  
Backup: ✅ Implementado

## Recomendações

1. Implementar criptografia de dados sensíveis (AES-256)
  2. Adicionar validação de integridade (checksums)
  3. Criar sistema de versionamento de schema
  4. Implementar auditoria completa de acesso
  5. Adicionar conformidade LGPD/GDPR
- 

## 2.8 Inventory System

### Pontos Fortes

- ✅ Gerenciamento de slots eficiente
- ✅ Suporte a stacking de itens
- ✅ Equipamento sincronizado
- ✅ UI responsiva

### Pontos Fracos

- ❌ Sem validação de operações no servidor
- ❌ Sem proteção contra item duplication
- ❌ Sem limite de peso/volume
- ❌ Falta de auditoria de movimentação



## Vulnerabilidades de Segurança

| Vulnerabilidade               | Severidade                                                                                | Impacto                    |
|-------------------------------|-------------------------------------------------------------------------------------------|----------------------------|
| Sem validação de movimentação |  Crítica | Duplication                |
| Sem limite de peso            |  Alta    | Exploração de espaço       |
| Sem auditoria                 |  Alta    | Impossível rastrear fraude |

## Recomendações

1. Implementar validação de todas as operações no servidor
  2. Adicionar sistema de peso/volume
  3. Criar auditoria de movimentação de itens
  4. Implementar proteção contra race conditions
- 

## 2.9 HUD System

### Pontos Fortes

-  Interface responsiva
-  Atualização eficiente
-  Suporte a múltiplas resoluções
-  Customização de layout

### Pontos Fracos

-  Sem cache de renderização
-  Sem otimização de batches
-  Sem sistema de priorização de updates
-  Falta de acessibilidade

## Performance

Tempo de renderização: 2-5ms  
Elementos na tela: ~150  
Batches de draw: ~30  
Memória de texturas: ~80MB

## Recomendações

1. Implementar canvas caching
  2. Otimizar batches de renderização
  3. Adicionar sistema de priorização de updates
  4. Implementar acessibilidade (WCAG 2.1)
- 

## 3. Análise de Escalabilidade

### 3.1 Escalabilidade Horizontal

#### Atual

- ✗ Sem suporte a múltiplos servidores
- ✗ Sem sistema de load balancing
- ✗ Sem sincronização entre instâncias

#### Recomendações

1. Implementar arquitetura de microserviços
2. Adicionar message broker (RabbitMQ/Kafka)
3. Criar sistema de sincronização distribuída
4. Implementar load balancing inteligente

## 3.2 Escalabilidade Vertical

### Atual

- ⚠️ Limitado a ~500 entidades simultâneas
- ⚠️ Uso de memória crescente
- ⚠️ Sem otimização de CPU

### Recomendações

- Otimizar estruturas de dados
- Implementar spatial partitioning (quadtree/octree)
- Adicionar LOD (Level of Detail)
- Implementar culling agressivo

## 3.3 Projeção de Crescimento

| Usuários Simultâneos | Servidores Necessários | Infraestrutura  |
|----------------------|------------------------|-----------------|
| 100                  | 1                      | Básica          |
| 1.000                | 3-5                    | Distribuída     |
| 10.000               | 15-20                  | Cloud escalável |
| 100.000              | 50+                    | Multi-região    |

## 4. Análise de Performance

### 4.1 Benchmarks Atuais

| Métrica          | Valor    | Alvo   | Status  |
|------------------|----------|--------|---------|
| FPS              | 60       | 60+    | ✔ OK    |
| Latência de Rede | 50-100ms | <100ms | ✔ OK    |
| Tempo de Load    | 15s      | <10s   | ⚠ Acima |
| Uso de RAM       | 450MB    | <400MB | ⚠ Acima |
| Uso de CPU       | 35%      | <30%   | ⚠ Acima |

### 4.2 Gargalos Identificados

- Garbage Collection** (45ms picos)
  - Impacto: Stuttering visível
  - Solução: Object pooling agressivo
- Renderização de HUD** (5ms)
  - Impacto: Redução de FPS
  - Solução: Canvas caching
- Sincronização de Rede** (100ms latência)
  - Impacto: Desincronização de estado
  - Solução: Interpolação/extrapolação
- Spawn de Entidades** (10ms por entidade)
  - Impacto: Lag ao entrar em áreas
  - Solução: Spawn assíncrono

### 4.3 Otimizações Recomendadas

| Prioridade | Otimização           | Ganho Esperado |
|------------|----------------------|----------------|
| 1          | GC tuning            | +15 FPS        |
| 2          | HUD cache            | +5 FPS         |
| 3          | Spatial partitioning | +20 FPS        |

|   |                      |
|---|----------------------|
| 4 | LOD system   +10 FPS |
| 5 | Async spawn   +8 FPS |

## 5. Análise de Segurança

### 5.1 Vulnerabilidades Críticas

#### 1. Validação Insuficiente no Servidor

- **Risco:** Cheating, exploração de mecânicas
- **Severidade:** 🔴 Crítica
- **Recomendação:** Implementar validação completa no servidor

#### 2. Falta de Criptografia de Dados

- **Risco:** Exposição de dados sensíveis
- **Severidade:** 🔴 Crítica
- **Recomendação:** Implementar AES-256 para dados sensíveis

#### 3. Sem Proteção contra Duplication

- **Risco:** Inflação econômica, economia quebrada
- **Severidade:** 🔴 Crítica
- **Recomendação:** Implementar validação transacional

#### 4. Sem Anti-Cheat

- **Risco:** Cheating generalizado
- **Severidade:** 🔴 Crítica
- **Recomendação:** Implementar sistema anti-cheat robusto

## 5.2 Vulnerabilidades Altas

| Vulnerabilidade        | Impacto                    | Solução                        |
|------------------------|----------------------------|--------------------------------|
| Sem rate limiting      | DoS                        | Implementar rate limiting      |
| Sem validação de input | Injection                  | Validar todos os inputs        |
| Sem HTTPS              | MITM                       | Forçar HTTPS                   |
| Sem autenticação forte | Account takeover           | Implementar 2FA                |
| Sem auditoria          | Impossível rastrear fraude | Implementar auditoria completa |

## 5.3 Checklist de Segurança

### Autenticação

- ❑ Senha com hash bcrypt/argon2
- ❑ 2FA implementado
- ❑ Session timeout
- ❑ Logout seguro

### Autorização

- ❑ RBAC implementado
- ❑ Validação de permissões
- ❑ Auditoria de acesso
- ❑ Princípio do menor privilégio

### Dados

- ❑ Criptografia em trânsito (HTTPS/TLS)
- ❑ Criptografia em repouso (AES-256)
- ❑ Backup criptografado
- ❑ Destruição segura de dados

### Aplicação

- ❑ Input validation
- ❑ Output encoding

- ❑ CSRF protection
- ❑ XSS protection
- ❑ SQL injection protection

#### Infraestrutura

- ❑ Firewall configurado
- ❑ WAF implementado
- ❑ DDoS protection
- ❑ Monitoramento de segurança

## 6. Conformidade com Boas Práticas

### 6.1 Padrões de Código

| Aspecto              | Status        | Observações                          |
|----------------------|---------------|--------------------------------------|
| Nomenclatura         | ✅ Consistente | Segue convenções C#                  |
| Documentação         | ⚠️ Parcial    | Faltam comentários em áreas críticas |
| Testes Unitários     | ❌ Ausentes    | Nenhum teste automatizado            |
| Testes de Integração | ❌ Ausentes    | Sem testes de integração             |
| Code Review          | ⚠️ Informal   | Sem processo formal                  |
| Versionamento        | ✅ Git         | Bem estruturado                      |

## 6.2 Arquitetura

| Aspecto                        | Status     | Observações                       |
|--------------------------------|------------|-----------------------------------|
| Separação de Responsabilidades | ⚠️ Parcial | Alguns sistemas acoplados         |
| DRY (Don't Repeat Yourself)    | ⚠️ Parcial | Código duplicado em alguns pontos |
| SOLID                          | ⚠️ Parcial | Nem todos os princípios aplicados |
| Design Patterns                | ✅ Bom      | Padrões bem utilizados            |
| Documentação de Arquitetura    | ❌ Ausente  | Sem documentação formal           |

## 6.3 DevOps e Deployment

| Aspecto           | Status    | Observações               |
|-------------------|-----------|---------------------------|
| CI/CD             | ❌ Ausente | Sem pipeline automatizado |
| Containerização   | ❌ Ausente | Sem Docker                |
| Orquestração      | ❌ Ausente | Sem Kubernetes            |
| Monitoramento     | ⚠️ Básico | Logging simples           |
| Alertas           | ❌ Ausente | Sem sistema de alertas    |
| Disaster Recovery | ❌ Ausente | Sem plano de recuperação  |



## 7. Recomendações de Melhoria

### 7.1 Curto Prazo (1-3 meses)

#### Segurança

##### 1. Implementar validação de combate no servidor

- Prioridade:  Crítica
- Esforço: 40 horas
- Impacto: Elimina cheating de dano

##### 2. Adicionar criptografia de dados sensíveis

- Prioridade:  Crítica
- Esforço: 30 horas
- Impacto: Protege dados de jogadores

##### 3. Implementar auditoria de ações críticas

- Prioridade:  Alta
- Esforço: 25 horas
- Impacto: Rastreabilidade de fraude

#### Performance

##### 4. Otimizar Garbage Collection

- Prioridade:  Alta
- Esforço: 20 horas
- Impacto: +15 FPS

##### 5. Implementar HUD caching

- Prioridade:  Alta
- Esforço: 15 horas
- Impacto: +5 FPS

#### Qualidade

##### 6. Criar testes unitários básicos

- Prioridade:  Alta
- Esforço: 35 horas
- Impacto: Reduz bugs

## 7.2 Médio Prazo (3-6 meses)

### Arquitetura

#### 1. Implementar Dependency Injection

- Prioridade: ● Alta
- Esforço: 50 horas
- Impacto: Reduz acoplamento

#### 2. Refatorar para microserviços

- Prioridade: ● Alta
- Esforço: 100 horas
- Impacto: Escalabilidade horizontal

#### 3. Implementar message broker

- Prioridade: ● Alta
- Esforço: 40 horas
- Impacto: Comunicação distribuída

### Performance

#### 4. Implementar spatial partitioning

- Prioridade: ● Alta
- Esforço: 60 horas
- Impacto: +20 FPS

#### 5. Adicionar LOD system

- Prioridade: ● Média
- Esforço: 45 horas
- Impacto: +10 FPS

### DevOps

#### 6. Implementar CI/CD pipeline

- Prioridade: ● Alta
- Esforço: 30 horas
- Impacto: Deployment automatizado

## 7.3 Longo Prazo (6-12 meses)

### Escalabilidade

#### 1. Implementar load balancing

- Prioridade: ● Alta
- Esforço: 80 horas
- Impacto: Suporta múltiplos servidores

#### 2. Adicionar replicação de dados

- Prioridade: ● Alta
- Esforço: 70 horas
- Impacto: Alta disponibilidade

#### 3. Implementar sharding de banco de dados

- Prioridade: ● Alta
- Esforço: 90 horas
- Impacto: Escalabilidade de dados

### Segurança

#### 4. Implementar anti-cheat avançado

- Prioridade: ● Alta
- Esforço: 120 horas
- Impacto: Reduz cheating

#### 5. Adicionar conformidade LGPD/GDPR

- Prioridade: ● Alta
- Esforço: 60 horas
- Impacto: Conformidade legal

### Qualidade

#### 6. Implementar testes de carga

- Prioridade: ● Média
  - Esforço: 50 horas
  - Impacto: Validação de escalabilidade
-

## 8. Plano de Ação Detalhado

### 8.1 Fase 1: Segurança Crítica (Semanas 1-4)

Semana 1-2: Validação de Combate no Servidor

- Implementar validação de range
- Implementar validação de cooldown
- Recalcular dano no servidor
- Adicionar logging de combate

Semana 2-3: Criptografia de Dados

- Implementar AES-256 para dados sensíveis
- Adicionar key management
- Migrar dados existentes
- Testes de criptografia

Semana 3-4: Auditoria de Ações

- Implementar logging de ações críticas
- Criar dashboard de auditoria
- Adicionar alertas de anomalias
- Testes de auditoria

### 8.2 Fase 2: Performance (Semanas 5-8)

Semana 5: GC Optimization

- Análise de alocações
- Implementar object pooling
- Tuning de GC
- Benchmarking

Semana 6: HUD Caching

- Implementar canvas caching
- Otimizar batches
- Testes de performance
- Benchmarking

Semana 7-8: Spatial Partitioning

- Implementar quadtree

- Integrar com spawn system
- Testes de performance
- Benchmarking

## 8.3 Fase 3: Arquitetura (Semanas 9-16)

### Semana 9-12: Dependency Injection

- Escolher framework (Zenject/Autofac)
- Refatorar Core System
- Refatorar outros sistemas
- Testes de integração

### Semana 13-16: Microserviços

- Definir limites de serviços
- Implementar message broker
- Migrar sistemas
- Testes de integração

## 9. Métricas de Sucesso

### 9.1 Segurança

| Métrica                   | Alvo  | Atual | Status |
|---------------------------|-------|-------|--------|
| Vulnerabilidades críticas | 0     | 4     | ✗      |
| Vulnerabilidades altas    | 0     | 5     | ✗      |
| Cobertura de auditoria    | 100%  | 20%   | ✗      |
| Conformidade LGPD/GDPR    | 100%  | 30%   | ✗      |
| Incidentes de segurança   | 0/mês | -     | -      |

### 9.2 Performance

| Métrica          | Alvo   | Atual    | Status |
|------------------|--------|----------|--------|
| FPS médio        | 60+    | 60       | ✓      |
| Latência de rede | <100ms | 50-100ms | ✓      |
| Tempo de load    | <10s   | 15s      | ✗      |

|             |  |        |  |       |  |   |
|-------------|--|--------|--|-------|--|---|
| Uso de RAM  |  | <400MB |  | 450MB |  | ✗ |
| Uso de CPU  |  | <30%   |  | 35%   |  | ✗ |
| Picos de GC |  | <10ms  |  | 45ms  |  | ✗ |

### 9.3 Escalabilidade

|                           |  |       |  |       |  |        |
|---------------------------|--|-------|--|-------|--|--------|
| Métrica                   |  | Alvo  |  | Atual |  | Status |
| Entidades simultâneas     |  | 1000+ |  | 500   |  | ✗      |
| Usuários simultâneos      |  | 1000+ |  | 100   |  | ✗      |
| Servidores suportados     |  | 10+   |  | 1     |  | ✗      |
| Latência de sincronização |  | <50ms |  | 100ms |  | ✗      |

### 9.4 Qualidade

|                      |  |      |  |       |  |        |
|----------------------|--|------|--|-------|--|--------|
| Métrica              |  | Alvo |  | Atual |  | Status |
| Cobertura de testes  |  | 80%  |  | 0%    |  | ✗      |
| Code review rate     |  | 100% |  | 50%   |  | ✗      |
| Bugs por release     |  | <5   |  | 15    |  | ✗      |
| Tempo de fix crítico |  | <24h |  | 48h   |  | ✗      |

---

## 10. Conclusões

### 10.1 Avaliação Geral

O Fields Online possui uma **arquitetura sólida como base**, com bons padrões de design e organização geral. No entanto, existem **vulnerabilidades críticas de segurança** que precisam ser endereçadas imediatamente antes de qualquer lançamento público.

**Pontuação Geral: 6.5/10**

| Aspecto             | Pontuação | Observação                     |
|---------------------|-----------|--------------------------------|
| Arquitetura         | 7/10      | Boa base, mas acoplada         |
| Segurança           | 3/10      | Crítica - requer ação imediata |
| Performance         | 6/10      | Aceitável, mas com gargalos    |
| Escalabilidade      | 4/10      | Limitada, requer refatoração   |
| Qualidade de Código | 7/10      | Bem organizado, sem testes     |
| DevOps              | 2/10      | Praticamente inexistente       |

## 10.2 Riscos Identificados

### ● Críticos

1. **Cheating generalizado** - Sem validação no servidor
2. **Duplication de itens** - Sem proteção transacional
3. **Exposição de dados** - Sem criptografia
4. **Impossibilidade de rastreamento** - Sem auditoria

### ● Altos

1. **Performance inadequada** - Gargalos de GC
2. **Escalabilidade limitada** - Arquitetura monolítica
3. **Falta de testes** - Sem cobertura automatizada
4. **Sem CI/CD** - Deployment manual e propenso a erros

## 10.3 Oportunidades

1. **Implementar validação no servidor** - Elimina cheating
  2. **Otimizar performance** - Melhora experiência do usuário
  3. **Refatorar para microserviços** - Permite escalabilidade
  4. **Adicionar testes** - Reduz bugs e aumenta confiabilidade
  5. **Implementar CI/CD** - Acelera desenvolvimento
-

## 11. Próximos Passos Recomendados

### 11.1 Imediato (Esta Semana)

1.  **Revisar este relatório com o time**
  - Apresentar achados principais
  - Discutir prioridades
  - Definir responsáveis
2.  **Criar backlog de segurança**
  - Priorizar vulnerabilidades críticas
  - Estimar esforço
  - Agendar sprints
3.  **Iniciar implementação de validação no servidor**
  - Começar com combat system
  - Criar testes de validação
  - Fazer deploy em staging

### 11.2 Curto Prazo (Próximas 2 Semanas)

1.  **Completar validação de combate**
  - Testes em staging
  - Feedback do time
  - Deploy em produção
2.  **Iniciar criptografia de dados**
  - Escolher algoritmo
  - Implementar key management
  - Começar migração
3.  **Criar plano de testes**
  - Definir estratégia de testes
  - Escolher framework
  - Começar testes unitários



## 11.3 Médio Prazo (Próximo Mês)

1.  **Completar todas as vulnerabilidades críticas**
  - Validação completa no servidor
  - Criptografia implementada
  - Auditoria funcional
2.  **Iniciar otimizações de performance**
  - GC tuning
  - HUD caching
  - Spatial partitioning
3.  **Implementar CI/CD básico**
  - Pipeline de build
  - Testes automatizados
  - Deployment automatizado

## 11.4 Longo Prazo (Próximos 3-6 Meses)

1.  **Refatorar para arquitetura escalável**
    - Implementar DI
    - Migrar para microserviços
    - Adicionar message broker
  2.  **Implementar anti-cheat avançado**
    - Detecção de anomalias
    - Análise comportamental
    - Sistema de reports
  3.  **Atingir conformidade legal**
    - LGPD/GDPR
    - Política de privacidade
    - Termos de serviço
-

## 12. Apêndices

### 12.1 Glossário de Termos

- **MMORPG:** Massively Multiplayer Online Role-Playing Game
- **DI:** Dependency Injection
- **CI/CD:** Continuous Integration/Continuous Deployment
- **LOD:** Level of Detail
- **GC:** Garbage Collection
- **LGPD:** Lei Geral de Proteção de Dados
- **GDPR:** General Data Protection Regulation
- **RBAC:** Role-Based Access Control
- **CSRF:** Cross-Site Request Forgery
- **XSS:** Cross-Site Scripting
- **WAF:** Web Application Firewall
- **DDoS:** Distributed Denial of Service

### 12.2 Referências e Recursos

#### **Segurança:**

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- CWE/SANS Top 25: <https://cwe.mitre.org/top25/>
- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>

#### **Performance:**

- Unity Performance Best Practices:  
<https://docs.unity3d.com/Manual/BestPracticeGuides.html>
- Game Engine Architecture: <https://www.gameenginebook.com/>

#### **Arquitetura:**

- Microservices Patterns: <https://microservices.io/>
- Clean Architecture: <https://blog.cleancoder.com/>

#### **DevOps:**

- Docker Documentation: <https://docs.docker.com/>
- Kubernetes Documentation: <https://kubernetes.io/docs/>

## 12.3 Contato e Suporte

**Responsável pela Auditoria:**

- Nome: Auditor Técnico
- Email: auditor@fieldsonline.com
- Telefone: +55 (XX) XXXX-XXXX

**Para Dúvidas ou Esclarecimentos:**

- Agendar reunião de follow-up
- Enviar email com questões específicas
- Consultar documentação técnica

---

## 13. Assinatura e Aprovação

| Papel              | Nome | Data       | Assinatura |
|--------------------|------|------------|------------|
| Auditor Técnico    | -    | 17/05/2026 | -          |
| Tech Lead          | -    | -          | -          |
| CTO                | -    | -          | -          |
| Gerente de Projeto | -    | -          | -          |

---

**Documento Confidencial - Uso Interno Apenas**

**Classificação: Confidencial**

**Data de Emissão: 17 de Maio de 2026**

**Versão: 1.0**